

SaaS Portfolio Developer

Documentazione Architeturale, Guida di Implementazione e Codice Sorgente (React + TypeScript + Firebase Realtime DB + Bootstrap)

React 18

TypeScript (TSX)

Firebase Auth

Realtime Database

Bootstrap 5

Vercel Deploy

Scopo del Documento: Questo manuale fornisce l'architettura completa, il codice TypeScript commentato e la guida passo-passo per assemblare e mettere online un SaaS per sviluppatore web junior. Il sistema include landing page dei servizi, autenticazione sicura, registrazione con ordine immediato del progetto, dashboard protetta, integrazione API esterna e regole di sicurezza per Firebase Realtime Database.

1. Struttura del Progetto e Architettura Cartelle

L'organizzazione segue i principi di modularità e separazione delle responsabilità (Separation of Concerns):

```
my-dev-saas/ |— public/ |— src/ | |— assets/ # Risorse statiche (Immagini, icone, logo) | |— components/ # Componenti UI riutilizzabili | | |— Navbar.tsx # Barra di navigazione dinamica | | |— ProtectedRoute.tsx # Guard per la protezione delle rotte | |— context/ # Gestione dello stato globale di autenticazione | | |— AuthContext.tsx | |— firebase/ # Configurazione e Inizializzazione SDK Firebase | | |— firebaseConfig.ts | |— models/ # Interfacce TypeScript e Modelli Dati | | |— ProjectModel.ts | |— pages/ # Pagine/Viste dell'applicazione | | |— Home.tsx # Landing Page (Servizi, Skill, Progetti) | | |— Login.tsx # Pagina di Login | | |— Register.tsx # Pagina di Registrazione | | |— Dashboard.tsx # Dashboard Protetta dell'utente | |— services/ # Modulo per chiamate API esterne (Axios) | | |— apiService.ts | |— App.tsx # Routing principale con React Router DOM | |— main.tsx # Entry point dell'applicazione React | |— index.css # Stili personalizzati e override Bootstrap |— .env.local # Variabili d'ambiente riservate (NON committare su Git) |— package.json |— vite.config.ts
```

2. Setup del Progetto e Installazione Dipendenze

Per creare il progetto da zero utilizzando Vite con il template React TypeScript, esegui i seguenti comandi nel terminale:

Terminale / Bash

```
# 1. Creazione del progetto con Vite + TypeScript
npm create vite@latest my-dev-saas -- --template react-ts

# 2. Accesso alla cartella del progetto
cd my-dev-saas

# 3. Installazione delle dipendenze necessarie
npm install firebase react-router-dom bootstrap react-bootstrap bootstrap-icons axios
```

3. Configurazione Firebase Realtime Database & Sicurezza

Di seguito è riportata la configurazione dell'SDK Firebase e le regole di sicurezza (Security Rules) da inserire nella console Firebase Realtime Database.

3.1 Variabili d'Ambiente (.env.local)

.env.local

```
VITE_FIREBASE_API_KEY=tua_api_key_qui
VITE_FIREBASE_AUTH_DOMAIN=tuo-progetto.firebaseio.com
VITE_FIREBASE_DATABASE_URL=https://tuo-progetto-default-rtdb.europe-west1.firebaseio.com
VITE_FIREBASE_PROJECT_ID=tuo-progetto
VITE_FIREBASE_STORAGE_BUCKET=tuo-progetto.appspot.com
VITE_FIREBASE_MESSAGING_SENDER_ID=123456789
VITE_FIREBASE_APP_ID=1:123456789:web:abcdef
```

3.2 Inizializzazione SDK (src/firebase/firebaseConfig.ts)

src/firebase/firebaseConfig.ts

```
import { initializeApp } from "firebase/app";
import { getAuth } from "firebase/auth";
import { getDatabase } from "firebase/database";

// Inizializzazione della configurazione mediante variabili d'ambiente protette
const firebaseConfig = {
  apiKey: import.meta.env.VITE_FIREBASE_API_KEY,
  authDomain: import.meta.env.VITE_FIREBASE_AUTH_DOMAIN,
  databaseURL: import.meta.env.VITE_FIREBASE_DATABASE_URL,
  projectId: import.meta.env.VITE_FIREBASE_PROJECT_ID,
  storageBucket: import.meta.env.VITE_FIREBASE_STORAGE_BUCKET,
  messagingSenderId: import.meta.env.VITE_FIREBASE_MESSAGING_SENDER_ID,
  appId: import.meta.env.VITE_FIREBASE_APP_ID
};

const app = initializeApp(firebaseConfig);

// Esportiamo i servizi di Autenticazione e Realtime Database
export const auth = getAuth(app);
export const db = getDatabase(app);
```

3.3 Regole di Sicurezza Realtime Database

Attenzione Sicurezza: Imposta queste regole nella scheda *Realtime Database* -> *Rules* della console Firebase per evitare accessi non autorizzati.

Firestore Realtime Database Rules (JSON)

```
{
  "rules": {
    "orders": {
      "$orderId": {
        // L'utente autenticato può leggere solo i propri ordini
        ".read": "auth != null && auth.uid === data.child('userId').val()",
        // L'utente autenticato può creare un ordine solo se assegna a sé stesso l'userId
        ".write": "auth != null && auth.uid === newData.child('userId').val()"
      }
    }
  }
}
```

4. Autenticazione e Rotte Protette (TypeScript)

4.1 AuthContext con Tipizzazione (src/context/AuthContext.tsx)

src/context/AuthContext.tsx

```
import React, { createContext, useContext, useEffect, useState, ReactNode } from 'react';
import { User, createUserWithEmailAndPassword, signInWithEmailAndPassword, signOut,
onAuthStateChanged } from 'firebase/auth';
import { auth } from '../firebase/firebaseConfig';

// Definizione dell'interfaccia per il contesto di autenticazione
interface AuthContextType {
  currentUser: User | null;
  signup: (email: string, password: string) => Promise<any>;
  login: (email: string, password: string) => Promise<any>;
  logout: () => Promise<void>;
}

const AuthContext = createContext<AuthContextType | undefined>(undefined);

export const useAuth = (): AuthContextType => {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error("useAuth deve essere utilizzato all'interno di un AuthProvider");
  }
  return context;
};

export const AuthProvider: React.FC<{ children: ReactNode }> = ({ children }) => {
  const [currentUser, setCurrentUser] = useState<User | null>(null);
  const [loading, setLoading] = useState<boolean>(true);

  const signup = (email: string, password: string) => createUserWithEmailAndPassword(auth, email,
password);
  const login = (email: string, password: string) => signInWithEmailAndPassword(auth, email, password);
  const logout = () => signOut(auth);

  useEffect(() => {
    const unsubscribe = onAuthStateChanged(auth, (user) => {
      setCurrentUser(user);
      setLoading(false);
    });
    return unsubscribe;
  }, []);

  return (
    <AuthContext.Provider value={{ currentUser, signup, login, logout }}>
      {!loading && children}
    </AuthContext.Provider>
  );
};
```

4.2 Componente Guard Rotte Protette (src/components/ProtectedRoute.tsx)

src/components/ProtectedRoute.tsx

```
import React from 'react';
import { Navigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';

interface ProtectedRouteProps {
  children: JSX.Element;
}

const ProtectedRoute: React.FC<ProtectedRouteProps> = ({ children }) => {
  const { currentUser } = useAuth();

  // Reindirizza al login se l'utente non è autenticato
  if (!currentUser) {
    return <Navigate to="/login" replace />;
  }

  return children;
};

export default ProtectedRoute;
```

5. Modelli Dati e Servizi API Esterni

5.1 Modello Progetto (src/models/ProjectModel.ts)

src/models/ProjectModel.ts

```
export interface Project {
  id: number;
  title: string;
  description: string;
  category: string;
  tags: string[];
  estimatedDays: number;
}

export const SAMPLE_PROJECTS: Project[] = [
  {
    id: 1,
    title: "E-Commerce Full Stack",
    description: "Piattaforma e-commerce reattiva con carrello integrato e gestione ordini.",
    category: "Web App",
    tags: ["React", "TypeScript", "Node.js"],
    estimatedDays: 14
  },
  {
    id: 2,
    title: "SaaS Dashboard Analitica",
    description: "Pannello di gestione aziendale con grafici e integrazioni API in tempo reale.",
    category: "SaaS",
    tags: ["React", "Bootstrap", "Firebase"],
    estimatedDays: 10
  },
  {
    id: 3,
    title: "Sito Web Aziendale & SEO",
    description: "Sito vetrina ad alte prestazioni, accessibile e ottimizzato per i motori di ricerca.",
    category: "Website",
    tags: ["HTML5", "Sass", "JavaScript"],
    estimatedDays: 5
  }
];
```

5.2 Servizio API (src/services/apiService.ts)

src/services/apiService.ts

```
import axios from 'axios';

const API_BASE_URL = 'https://jsonplaceholder.typicode.com';

// Recupera i dati simulati per il feed delle attività in lavorazione
export const getDeveloperStats = async (): Promise<any[]> => {
  try {
    const response = await axios.get(`${API_BASE_URL}/todos?_limit=5`);
    return response.data;
  } catch (error) {
    console.error("Errore durante il recupero dei dati dall'API esterna:", error);
    throw error;
  }
};
```

6. Interfacce Utente e Pagine (TSX)

6.1 Landing Page (src/pages/Home.tsx)

src/pages/Home.tsx

```

import React from 'react';
import { Container, Row, Col, Card, Button, Badge } from 'react-bootstrap';
import { useNavigate } from 'react-router-dom';
import { SAMPLE_PROJECTS, Project } from '../models/ProjectModel';
import { useAuth } from '../context/AuthContext';

const Home: React.FC = () => {
  const navigate = useNavigate();
  const { currentUser } = useAuth();

  const handleSelectProject = (projectTitle: string) => {
    if (!currentUser) {
      // Reindirizza alla registrazione portando con sé la scelta del progetto
      navigate('/register', { state: { selectedProject: projectTitle } });
    } else {
      // Se già autenticato, va direttamente in Dashboard per confermare l'ordine
      navigate('/dashboard', { state: { selectedProject: projectTitle } });
    }
  };

  return (
    <Container className="py-5">
      <div className="text-center mb-5">
        <h1 className="display-4 fw-bold text-primary">Sviluppo Web Moderno & Sicuro</h1>
        <p className="lead text-muted">Soluzioni Web Full-Stack, SaaS e API su misura per la tua
attività.</p>
      </div>

      <section className="mb-5">
        <h3 className="text-center mb-4">Tecnologie & Competenze</h3>
        <div className="d-flex justify-content-center gap-2 flex-wrap">
          {[
            "React", "TypeScript", "Bootstrap 5", "Firebase Realtime DB", "REST API", "Git &
Security"
          ].map((tech, index) => (
            <Badge key={index} bg="dark" className="p-3 fs-6">{tech}</Badge>
          ))}
        </div>
      </section>

      <section>
        <h3 className="text-center mb-4">Progetti e Servizi Disponibili</h3>
        <Row>
          {SAMPLE_PROJECTS.map((project: Project) => (
            <Col md={4} key={project.id} className="mb-4">
              <Card className="h-100 shadow-sm border-0">
                <Card.Body className="d-flex flex-column">
                  <Card.Title className="fw-bold">{project.title}</Card.Title>
                  <Card.Text className="text-muted">{project.description}</Card.Text>
                  <div className="mb-3">
                    {project.tags.map((tag, i) => (
                      <Badge bg="secondary" className="me-1" key={i}>{tag}</Badge>
                    ))}
                  </div>
                  <div className="mt-auto">
                    <p className="small text-muted mb-2">Stima consegna: ~{project.estimatedDays}
giorni</p>
                    <Button
                      variant="primary"
                      className="w-100"
                      onClick={() => handleSelectProject(project.title)}
                    >
                      Richiedi Questo Servizio
                    </Button>
                  </div>
                </Card.Body>
              </Card>
            </Col>
          ))}
        </Row>
      </section>
    </Container>
  );
};

```

```
        </Row>
      </section>
    </Container>
  );
};

export default Home;
```

6.2 Pagina di Registrazione (src/pages/Register.tsx)

```
src/pages/Register.tsx
```

```

import React, { useState } from 'react';
import { Container, Card, Form, Button, Alert } from 'react-bootstrap';
import { useAuth } from '../context/AuthContext';
import { useNavigate, useLocation, Link } from 'react-router-dom';

const Register: React.FC = () => {
  const [email, setEmail] = useState<string>('');
  const [password, setPassword] = useState<string>('');
  const [error, setError] = useState<string>('');
  const [loading, setLoading] = useState<boolean>(false);

  const { signup } = useAuth();
  const navigate = useNavigate();
  const location = useLocation();

  const selectedProject = location.state?.selectedProject || null;

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    try {
      setError('');
      setLoading(true);
      await signup(email, password);
      navigate('/dashboard', {
        state: {
          selectedProject,
          feedback: 'Account creato con successo! Il tuo ordine è stato registrato.'
        }
      });
    } catch (err: any) {
      setError('Errore durante la registrazione: ' + err.message);
    } finally {
      setLoading(false);
    }
  };

  return (
    <Container className="d-flex align-items-center justify-content-center" style={{ minHeight:
'80vh' }}>
      <Card style={{ maxWidth: '420px', width: '100%' }} className="p-4 shadow border-0">
        <Card.Body>
          <h2 className="text-center mb-4 fw-bold">Crea Account</h2>

          {selectedProject && (
            <Alert variant="info" className="small">
              Stai completando la registrazione per ordinare: <strong>{selectedProject}</strong>
            </Alert>
          )}

          {error && <Alert variant="danger">{error}</Alert>}

          <Form onSubmit={handleSubmit}>
            <Form.Group className="mb-3" controlId="email">
              <Form.Label>Email</Form.Label>
              <Form.Control
                type="email"
                required
                value={email}
                onChange={(e) => setEmail(e.target.value)}
              />
            </Form.Group>

            <Form.Group className="mb-3" controlId="password">
              <Form.Label>Password</Form.Label>
              <Form.Control
                type="password"
                required

```



```

        value={password}
        onChange={(e) => setPassword(e.target.value)}
      />
    </Form.Group>

    <Button disabled={loading} className="w-100" type="submit" variant="primary">
      Registrati e Prosegui
    </Button>
  </Form>

  <div className="text-center mt-3 small text-muted">
    Hai già un account? <Link to="/login">Accedi qui</Link>
  </div>
</Card.Body>
</Card>
</Container>
);
};

export default Register;

```

6.3 Pagina di Login (src/pages/Login.tsx)

src/pages/Login.tsx

```

import React, { useState } from 'react';
import { Container, Card, Form, Button, Alert } from 'react-bootstrap';
import { useAuth } from '../context/AuthContext';
import { useNavigate, Link } from 'react-router-dom';

const Login: React.FC = () => {
  const [email, setEmail] = useState<string>('');
  const [password, setPassword] = useState<string>('');
  const [error, setError] = useState<string>('');
  const [loading, setLoading] = useState<boolean>(false);

  const { login } = useAuth();
  const navigate = useNavigate();

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    try {
      setError('');
      setLoading(true);
      await login(email, password);
      navigate('/dashboard');
    } catch (err: any) {
      setError('Credenziali errate o utente non trovato.');
```

6.4 Dashboard Utente con Realtime DB & API (src/pages/Dashboard.tsx)

src/pages/Dashboard.tsx

```

import React, { useState, useEffect } from 'react';
import { Container, Row, Col, Card, Alert, Button, ListGroup, Spinner } from 'react-bootstrap';
import { useAuth } from '../context/AuthContext';
import { useLocation, useNavigate } from 'react-router-dom';
import { db } from '../firebase/firebaseConfig';
import { ref, push, set, onValue } from 'firebase/database';
import { getDeveloperStats } from '../services/apiService';

interface Order {
  id: string;
  projectName: string;
  status: string;
  userId: string;
}

const Dashboard: React.FC = () => {
  const { currentUser, logout } = useAuth();
  const location = useLocation();
  const navigate = useNavigate();

  const [feedback, setFeedback] = useState<string>(location.state?.feedback || '');
  const [orders, setOrders] = useState<Order[]>([]);
  const [apiData, setApiData] = useState<any[]>([]);
  const [loadingApi, setLoadingApi] = useState<boolean>(true);

  // 1. Invio del progetto selezionato su Realtime Database
  useEffect(() => {
    const selectedProject = location.state?.selectedProject;

    if (selectedProject && currentUser) {
      const saveOrder = async () => {
        try {
          const ordersRef = ref(db, 'orders');
          const newOrderRef = push(ordersRef);

          await set(newOrderRef, {
            userId: currentUser.uid,
            userEmail: currentUser.email,
            projectName: selectedProject,
            status: 'In Elaborazione',
            createdAt: new Date().toISOString()
          });

          setFeedback(`Richiesta inviata con successo per il servizio: ${selectedProject}!`);
        } catch (error) {
          console.error("Errore nel salvataggio dell'ordine:", error);
        }
      };
      saveOrder();
    }
  }, [location.state, currentUser]);

  // 2. Lettura in tempo reale degli ordini associati all'utente
  useEffect(() => {
    if (!currentUser) return;

    const ordersRef = ref(db, 'orders');

    const unsubscribe = onValue(ordersRef, (snapshot) => {
      const data = snapshot.val();
      if (data) {
        const userOrders: Order[] = Object.keys(data)
          .map(key => ({
            id: key,
            ...data[key]
          }))
          .filter(order => order.userId === currentUser.uid);
      }
    });
  }, [currentUser]);

```

```

        setOrders(userOrders);
    } else {
        setOrders([]);
    }
  });

  return () => unsubscribe();
}, [currentUser]);

// 3. Recupero dei dati dall'API esterna
useEffect(() => {
  getDeveloperStats()
    .then(data => {
      setApiData(data);
      setLoadingApi(false);
    })
    .catch(() => setLoadingApi(false));
}, []);

const handleLogout = async () => {
  await logout();
  navigate('/login');
};

return (
  <Container className="py-5">
    <div className="d-flex justify-content-between align-items-center mb-4">
      <h2 className="fw-bold">Dashboard Utente</h2>
      <Button variant="outline-danger" onClick={handleLogout}>Logout</Button>
    </div>

    <p className="text-muted">Utente autenticato: <strong>{currentUser?.email}</strong></p>

    {feedback && <Alert variant="success" dismissible onClose={() => setFeedback('')}>{feedback}</Alert>}

    <Row className="mt-4">
      <Col md={6} className="mb-4">
        <Card className="shadow-sm border-0 h-100">
          <Card.Header className="bg-primary text-white fw-bold">I Tuoi Servizi Ordinati</Card.Header>
          <Card.Body>
            {orders.length === 0 ? (
              <p className="text-muted">Non hai ancora richiesto nessun servizio.</p>
            ) : (
              <ListGroup variant="flush">
                {orders.map((order) => (
                  <ListGroup.Item key={order.id} className="d-flex justify-content-between align-items-center">
                    <div><strong>{order.projectName}</strong></div>
                    <span className="badge bg-warning text-dark">{order.status}</span>
                  </ListGroup.Item>
                ))}
              </ListGroup>
            )}
          </Card.Body>
        </Card>
      </Col>

      <Col md={6} className="mb-4">
        <Card className="shadow-sm border-0 h-100">
          <Card.Header className="bg-dark text-white fw-bold">Live API Feed (Task in Lavorazione)</Card.Header>
          <Card.Body>
            {loadingApi ? (
              <div className="text-center py-3">
                <Spinner animation="border" size="sm" /> Caricamento API...
              </div>
            ) : (
              <div>
                <strong>{apiData.projectName}</strong>
                <span className="badge bg-warning text-dark">{apiData.status}</span>
              </div>
            )}
          </Card.Body>
        </Card>
      </Col>
    </Row>
  </Container>
);

```

```

        </div>
      ) : (
        <ListGroup variant="flush">
          {apiData.map((task: any) => (
            <ListGroup.Item key={task.id} className="small">
              <i className={`bi ${task.completed ? 'bi-check-circle-fill text-success' : 'bi-clock text-secondary'} me-2`}></i>
              {task.title}
            </ListGroup.Item>
          )))}
        </ListGroup>
      )}
    </Card.Body>
  </Card>
</Col>
</Row>
</Container>
);
};

export default Dashboard;

```

6.5 Navigazione e Configurazione Rotte (src/App.tsx)

src/App.tsx

```

import React from 'react';
import { BrowserRouter as Router, Routes, Route, Link } from 'react-router-dom';
import { Navbar, Nav, Container } from 'react-bootstrap';
import { AuthProvider, useAuth } from '../context/AuthContext';
import Home from '../pages/Home';
import Login from '../pages/Login';
import Register from '../pages/Register';
import Dashboard from '../pages/Dashboard';
import ProtectedRoute from '../components/ProtectedRoute';

const NavigationBar: React.FC = () => {
  const { currentUser } = useAuth();

  return (
    <Navbar bg="dark" variant="dark" expand="lg">
      <Container>
        <Navbar.Brand as={Link} to="/" className="fw-bold text-primary">DevSaaS Portfolio</Navbar.Brand>
        <Navbar.Toggle aria-controls="basic-navbar-nav" />
        <Navbar.Collapse id="basic-navbar-nav">
          <Nav className="ms-auto">
            <Nav.Link as={Link} to="/">Home</Nav.Link>
            {currentUser ? (
              <Nav.Link as={Link} to="/dashboard">Dashboard</Nav.Link>
            ) : (
              <>
                <Nav.Link as={Link} to="/login">Login</Nav.Link>
                <Nav.Link as={Link} to="/register">Registrati</Nav.Link>
              </>
            )}
          </Nav>
        </Navbar.Collapse>
      </Container>
    </Navbar>
  );
};

function App() {
  return (
    <AuthProvider>
      <Router>
        <NavigationBar />
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/login" element={<Login />} />
          <Route path="/register" element={<Register />} />
          <Route
            path="/dashboard"
            element={
              <ProtectedRoute>
                <Dashboard />
              </ProtectedRoute>
            }
          />
        </Routes>
      </Router>
    </AuthProvider>
  );
}

export default App;

```

7. Guida al Deploy Gratuito su Vercel

Segui questi tre passaggi per ospitare gratuitamente la tua web app su Vercel:

Passo	Operazione	Dettagli / Comandi
1	Caricamento su GitHub	Inizializza la repository Git e pubblica il codice su GitHub: <code>git init && git add . && git commit -m "Initial commit SaaS"</code>
2	Importazione Vercel	Accedi a vercel.com , crea un nuovo progetto e importa la repository. Configura le Environment Variables inserendo le chiavi <code>VITE_FIREBASE_*</code> .
3	Abilitazione Dominio Firebase	Nella Console Firebase > <i>Authentication</i> > <i>Settings</i> > <i>Authorized domains</i> , aggiungi il dominio Vercel generato (es. <code>tuo-saas.vercel.app</code>).